

Solución a Prueba Técnica para Desarrollador Full Stack

Objetivo: Desarrollo de Red Social con Node.js y React

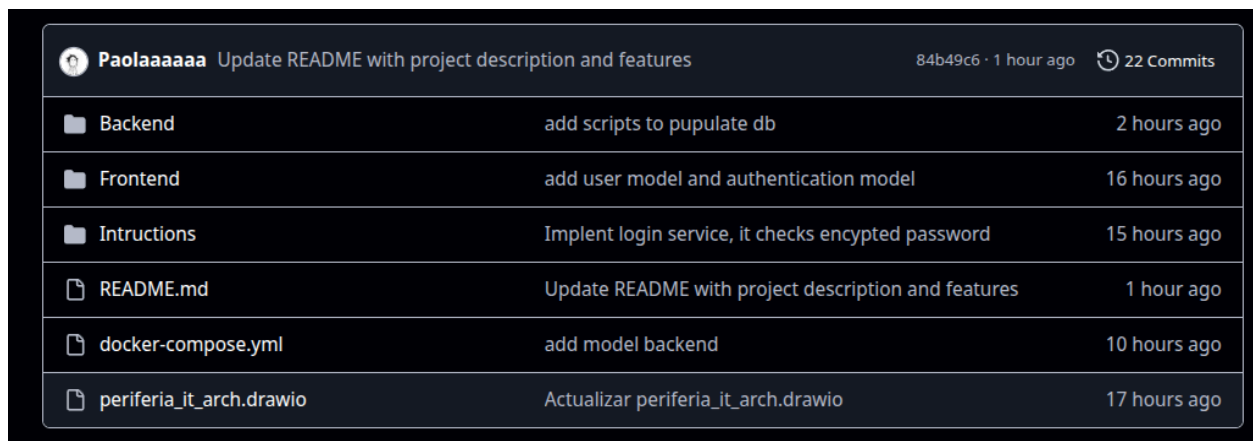
Paola Andrea Campiño, paocampino76@gmail.com

Link al repositorio: <https://github.com/Paolaaaaaa/speak>

En este documento puede encontrar la explicación y el instructivo para realizar el despliegue del backend de la red social desarrollada “Speak”. Esta red social tiene una arquitectura de microservicios, la cual consiste en dos servicios “Authenticator” y “Publications”.

01. Explicación de Propuesta :

El repositorio cuenta con dos carpetas una para el **Backend** y otra para el **Frontend**, dentro del backend se encuentran dos proyectos uno para Authenticator y otro para Publications. Cabe resaltar que **docker compose** se encuentra en la carpeta raíz.



 Paolaaaaaa	Update README with project description and features	84b49c6 · 1 hour ago	 22 Commits
 Backend	add scripts to pupulate db		2 hours ago
 Frontend	add user model and authentication model		16 hours ago
 Intructions	Implent login service, it checks encrypted password		15 hours ago
 README.md	Update README with project description and features		1 hour ago
 docker-compose.yml	add model backend		10 hours ago
 periferia_it_arch.drawio	Actualizar periferia_it_arch.drawio		17 hours ago

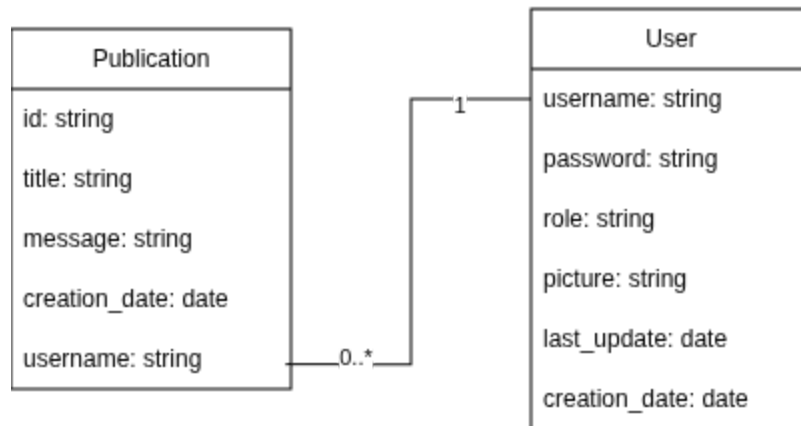
Speak es una red-social, la cual cuenta con funcionalidades como:

- Hacer login
- Crear publicaciones
- Ver publicaciones
- Listar aplicaciones

En base a lo anterior el entendimiento que se da es que un usuario puede tener 0 a muchas publicaciones, pero una publicación solo puede tener asociado un usuario publicador.

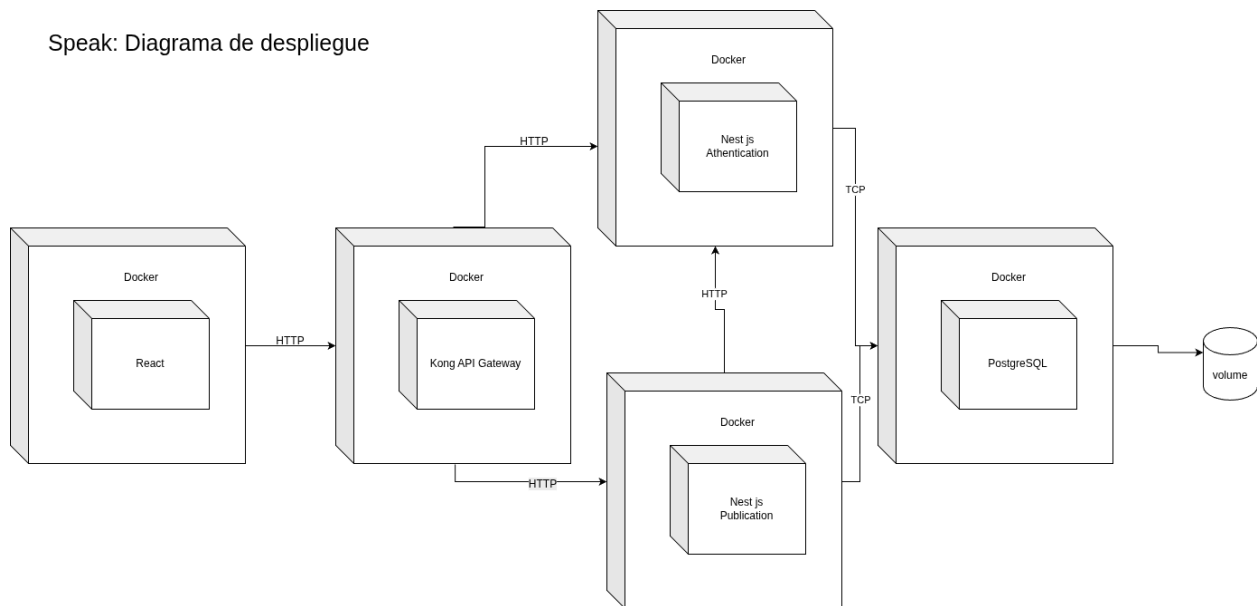
A continuación se ve un diagrama con el entendimiento del problema:

Entendimiento del Mundo del problema



De acuerdo con las necesidades especificadas la aplicación debe tener una arquitectura de microservicios y se debe cumplir con las funcionalidades previamente especificadas. En base a lo anterior se propone tener dos servicios: Uno dedicado a la autenticación e información del usuario y otro dedicado al manejo de las publicaciones. Ambos servicios van a usar la misma base de datos. A continuación se puede ver más detalle cómo se realizaría el despliegue de los diferentes componentes del sistema:

Speak: Diagrama de despliegue



En el diagrama se puede ver de izquierda a derecha:

- Un frontend que va a ser desplegado en un contenedor y desarrollado con React
- un contenedor con Kong API gateway para el manejo unificado de peticiones al back desde el frontend.
- Tenemos dos contenedores para los dos microservicios, Authentication
- La base de datos desplegada en docker con un volumen adjunto para permitir la persistencia de la información.

02. Instructivo para despliegue:

- Asegúrese de tener docker daemon esté corriendo
- Esta acción se puede realizar abriendo la aplicación de docker escritorio
- Clone el repositorio con el siguiente comando:

```
git clone git@github.com:Paolaaaaa/speak.git
```

- Use docker compose para levantar todos los servicios por primera vez ejecute el comando a continuación en la carpeta speak:

```
docker Compose up --build
```

- Revise que todos los servicios se encuentren desplegados correctamente, mediante el comando :

```
docker ps
```

03. Instructivo para población de Base de datos:

Asegúrese de haber seguido los pasos [anteriores](#) para realizar el despliegue y que todos los servicios estén corriendo. Para poblar la base de datos vamos a ejecutar un script .sh.

- a. De permisos de ejecución al script que tiene como nombre populate users.sh y ha populate_publications.sh

```
chmod +x Backend/scripts/populate_db_test1.sh
```

- b. Ejecute el script:

```
Backend/scripts/populate_db_test1.sh
```

- c. Si tiene herramientas como Dbeaver o PgAdmin podrá observar que ya la base de datos está poblada.

04. Instructivo para Ejecutar frontend:

1. Desde la carpeta raíz diríjase a la carpeta Frontend.

```
Cd Frontend/
```

2. Use el gestor de paquetes npm para instalar las dependencias

```
npm install
```

3. Ejecute el project modo desarrollo.

```
npm run dev
```